Highlights

Revisiting Kernel Search for the Large-Scale Multi-Source Capacitated Facility Location Problem: Three Targeted Design Improvements

Kamal Lamsal

- We revisit Kernel Search for the large-scale Multi-Source CFLP and separate its runtime into LP and restricted-MILP components.
- Column generation targets the LP-relaxation bottleneck while preserving the LP-derived information used by KS.
- A flat-k edge rule gives explicit base edge-set sizes across instance families and capacity regimes.
- Preserving incumbent arcs between stages makes the previous incumbent feasible in the next restricted MILP.
- The computational study is structured to isolate LP acceleration, MILP-stage acceleration, and end-to-end performance.

Revisiting Kernel Search for the Large-Scale Multi-Source Capacitated Facility Location Problem: Three Targeted Design Improvements

Kamal Lamsal[a]

[a]*Department of Computer Information Systems and Business Analytics, James Madison University, Harrisonburg, Virginia, USA*

Abstract

Kernel Search (KS) remains one of the strongest matheuristics for the large-scale Multi-Source Capacitated Facility Location Problem (MSCFLP). This paper takes a design-analysis perspective on G&S 2012 Kernel Search. A close examination of the original method reveals three design choices that can each be refined independently, yielding consistent improvements in both solution quality and computation time.

The first concerns LP construction cost: the full LP relaxation contains all $|\mathcal{J}||\mathcal{I}|$ shipping variables, most of which play no role in the optimal solution. Column generation recovers the same bound, facility partition, and reduced-cost signal at substantially lower cost.

The second concerns arc selection: the global reduced-cost threshold γ used to build restricted MILPs provides no size guarantee and systematically admits fewer arcs for bucket facilities than for kernel facilities, because bucket facilities have LP dual zero and receive no capacity-cost discount on their arc reduced costs.

The third concerns stage transitions: when the arc set changes between stages, the incumbent solution from the previous subproblem may no longer be feasible in the next, because arcs it uses may be absent. The solver must rediscover feasibility from scratch at every stage transition.

We address each with a targeted modification. Column generation replaces full LP construction. A client-centric flat-k criterion replaces the global γ threshold, selecting exactly k arcs per client by reduced cost and treating

Email address: lamsalkx@jmu.edu (Kamal Lamsal)

all facilities uniformly. Stage transitions are made feasibility-preserving by including every arc used by the current incumbent when building the next edge set; we prove that this guarantees the incumbent is feasible in the next restricted MILP by construction, making it a valid warm start at every stage. The resulting repaired method retains the full architecture of G&S 2012 and is named *Funnel Kernel Search* (FKS).

The computational study isolates each modification via a dedicated comparison, so that each improvement can be attributed to a specific design decision. Results are reported across five large-scale benchmark families from the Avella et al. test bed. FKS finds the proven optimal solution on 109 of the 110 Test Bed A instances for which an optimum is known, demonstrating that the method is not only faster but also produces solutions of extremely high quality.

Keywords: Capacitated facility location, Kernel search, Column generation, Matheuristic, LP relaxation, Feasibility preservation, Multi-source

---

1. Introduction

The *Capacitated Facility Location Problem* (CFLP) is a classical model for selecting a set of facilities and allocating customer demand to them under capacity restrictions. It appears in distribution network design, supply chain planning, emergency service location, and telecommunications infrastructure design (Klose and Drexl, 2005; Daskin, 2011; Laporte et al., 2019). In the *Multi-Source* CFLP (MSCFLP), a customer's demand may be split among several open facilities. In the *Single-Source* CFLP (SSCFLP), each customer must be assigned to one facility; this additional assignment restriction substantially changes the structure and computational difficulty of the problem (Klose and Drexl, 2005). Both variants are NP-hard (Sridharan, 1995).

This paper focuses on the large-scale MSCFLP. Avella et al. (Avella et al., 2008) established a widely used computational benchmark for this setting and proposed a Lagrangean-relaxation and core-problem heuristic for instances with millions of variables. Guastaroba and Speranza (Guastaroba and Speranza, 2012) introduced a Kernel Search (KS) matheuristic for the same problem class (which they refer to simply as the CFLP; we retain the MSCFLP label to distinguish it from the single-source variant). Their method solves the LP relaxation, identifies an initial kernel of LP-open and LP-fractional facilities, and solves a sequence of restricted MILPs that

add promising LP-closed facilities through capacity-expanding buckets. The approach is effective because the strong MSCFLP formulation includes variable upper-bound constraints, which make the LP solution a reliable predictor of the facilities and arcs used in high-quality integer solutions. Speed controls in the original paper act on the restricted-MILP phase: bucket count limits, variable-count caps per subproblem, a kernel-pruning rule based on repeated non-selection, and a forcing constraint requiring each bucket subproblem to use a facility from the current bucket. A faster variant, B-KS(0), searches only over the initial LP-selected kernel. The original paper frames KS performance as a balance between restricted-search quality and the time required to solve the resulting MILPs.

On the exact-method side, Görtz and Klose (Görtz and Klose, 2012) proposed a Lagrangian branch-and-bound that is competitive with the best exact approaches on the smaller GK instances. Fischetti, Ljubić, and Sinnl (Fischetti et al., 2016) later showed that a modern Benders decomposition embedded in a branch-and-cut solver proves optimality for 210 out of 445 large-scale p* instances within a 50,000-second time limit, and consistently produces smaller gaps than KS across all three test beds. Their computational study confirms that solving the LP at the root node dominates the total runtime for the largest instances: more than 90% of overall computing time may be spent in LP-based cut separation. To the best of our knowledge, these are the only results available for exact methods on the large-scale multi-source CFLP benchmark of Avella et al.

This paper takes a different starting point: rather than proposing a new acceleration strategy, we ask whether the design choices embedded in G&S 2012 are each well-founded. A close reading of the method reveals three design choices, each of which can be refined independently.

LP construction cost. The full LP relaxation contains $|\mathcal{J}||\mathcal{I}|$ shipping variables. At optimality, most carry positive reduced cost and contribute nothing to the LP solution or to the KS guidance it produces. Constructing the full matrix is therefore unnecessary: column generation recovers the same bound, facility partition, and reduced-cost signal without ever materialising the irrelevant variables.

Arc selection bias. G&S 2012 includes arc (j, i) whenever its reduced cost falls below a global threshold γ, the mean reduced cost over LP-active arcs. The threshold provides no guarantee on subproblem size and is structurally biased: a bucket facility $j \in J_0$ has LP dual $\alpha_j^* = 0$, so its arc reduced cost $c_{ji} - \pi_i^*$ contains no capacity signal. A kernel facility with positive dual

receives a capacity-dual discount that lowers its reduced costs; the same γ therefore admits more kernel arcs than bucket arcs, giving bucket facilities a systematically smaller arc set in every restricted MILP.

Cold-start stage transitions. G&S 2012 passes no solution information between consecutive restricted MILPs. When the arc set is modified between stages, the incumbent from the previous subproblem may no longer be feasible: arcs it uses may simply be absent from the new edge set. Because feasibility cannot be guaranteed, the incumbent cannot be supplied as a warm start, and the solver must rediscover a feasible solution from scratch at every stage. This is not an obvious limitation of warm starting in general; it is a structural consequence of arc-set transitions that discard incumbent arcs.

We address each in turn. The LP is solved by column generation. Arc selection is replaced by a client-centric flat-k criterion: for each client, the k kernel facilities with lowest reduced cost are included, giving an exact subproblem-size guarantee and treating all facilities uniformly regardless of their LP dual value. Stage transitions are made feasibility-preserving by including every arc used by the current incumbent when building the next stage's edge set; we prove that this construction guarantees the incumbent is feasible in the next restricted MILP, making it a valid warm start by construction. The resulting repaired variant is *Funnel Kernel Search* (FKS). It retains the full architecture of G&S 2012 — the strong MSCFLP formulation, the LP-defined kernel, reduced-cost guidance, and bucket expansion — and modifies only the three components identified above.

The contributions are as follows.

- **Design analysis.** We examine the G&S 2012 design and identify three aspects where targeted modifications yield consistent improvements: (1) the LP is built in full when column generation recovers the same information at lower cost; (2) the global γ threshold for arc selection admits no size guarantee and treats bucket facilities less favorably, since their LP dual is zero and their arc reduced costs carry no capacity signal, causing the same γ to admit fewer arcs than for kernel facilities; (3) the restricted-MILP phase starts cold at each stage, forcing the solver to rediscover feasibility after every arc-set transition.

- **Fix 1 — LP cost.** We apply column generation to the MSCFLP LP relaxation, retaining the bound, facility partition, and reduced-cost signal required by KS while avoiding full-matrix construction.

- **Fix 3 — feasibility-preserving stage transitions.** We prove that including every incumbent arc in the next stage's edge set guarantees the previous solution is feasible in the next restricted MILP (Proposition 1). This is a structural result, not a heuristic: feasibility is guaranteed by construction and makes the incumbent a provably valid MIP start at every stage.

- **Fix 2 — arc selection.** With the feasibility guarantee in place, we introduce a client-centric flat-k criterion that replaces the global γ threshold. The flat-k rule narrows the fill set aggressively across stages ($50 \to 20 \to 10$ arcs per client) without risking cold restarts, because Proposition 1 holds regardless of how small k becomes. It also gives an exact subproblem-size guarantee (Proposition 2) and treats all facilities uniformly.

- **Component-wise study.** The computational study isolates each fix via a dedicated comparison: full LP versus CG LP (Fix 1 alone), KS-CG versus KS-CG-WS (Fix 3 alone), KS-CG-WS versus FKS-CG-WS (Fix 2 alone), and end-to-end KS-full versus FKS-CG-WS (combined effect).

The result is a matheuristic that is not only faster than the original but also consistently finds solutions of higher quality. On the 110 Test Bed A instances with known proven optima, FKS finds the optimal solution on 109.

The remainder of the paper is organized as follows. Section 2 gives the MSCFLP formulation and the LP structure used by KS. Section 3 examines these three design aspects formally, then presents the modifications in order of dependency: column-generation LP (Fix 1), feasibility-preserving stage transitions with their structural proof (Fix 3), and flat-k arc selection (Fix 2) — which relies on Fix 3 to narrow arc sets safely. Section 4 describes the component-wise computational study and reports results on the large-scale Test Bed benchmarks. Section 5 concludes.

2. Problem Formulation and LP Structure

2.1. The Multi-Source CFLP

Let $\mathcal{J} = \{1, \ldots, m\}$ be the set of potential facility locations and $\mathcal{I} = \{1, \ldots, n\}$ the set of clients. Each facility $j \in \mathcal{J}$ has a fixed opening cost $f_j > 0$ and a capacity $q_j > 0$. Each client $i \in \mathcal{I}$ has a demand $d_i > 0$. The unit shipping cost from facility j to client i is $c_{ji} \geq 0$.

Let $y_j \in \{0,1\}$ indicate whether facility j is open, and let $x_{ji} \in [0, d_i]$ be the amount of client i's demand served by facility j. The MSCFLP is:

$$\min \quad \sum_{j\in\mathcal{J}} f_j y_j + \sum_{j\in\mathcal{J}}\sum_{i\in\mathcal{I}} c_{ji} x_{ji} \tag{1}$$

$$\text{s.t.} \quad \sum_{j\in\mathcal{J}} x_{ji} = d_i \qquad \forall i \in \mathcal{I} \tag{2}$$

$$\sum_{i\in\mathcal{I}} x_{ji} \le q_j y_j \qquad \forall j \in \mathcal{J} \tag{3}$$

$$x_{ji} \le d_i y_j \qquad \forall j \in \mathcal{J},\ i \in \mathcal{I} \tag{4}$$

$$y_j \in \{0,1\} \qquad \forall j \in \mathcal{J} \tag{5}$$

$$x_{ji} \ge 0 \qquad \forall j \in \mathcal{J},\ i \in \mathcal{I}. \tag{6}$$

Constraints (2) ensure all demand is satisfied. Constraints (3) enforce facility capacities. The variable-upper-bound (VUB) constraints (4) prevent routing demand to closed facilities. Together, constraints (3) and (4) ensure that $y_j = 0$ implies $x_{ji} = 0$ for all i, tightening the LP relaxation substantially compared to formulations without VUB constraints.

The LP relaxation is obtained by replacing (5) with $0 \le y_j \le 1$. We denote its optimal solution by $(x^{\mathrm{LP}}, y^{\mathrm{LP}})$ and its objective value by z^{LP}. Unless explicitly stated otherwise, gaps reported in this paper are measured relative to z^{LP}:

$$\mathrm{gap}(\%) = \frac{z^H - z^{\mathrm{LP}}}{z^{\mathrm{LP}}} \times 100,$$

where z^H is the heuristic objective. This gives a common lower-bound-based measure across all algorithmic variants. When comparing with published tables, we also report objective values because published gaps may use different lower bounds or hardware.

2.2. LP Solution Structure

The LP solution partitions facilities into three disjoint sets based on y_j^{LP}:

- $J_1 = \{j \in \mathcal{J} : y_j^{\mathrm{LP}} = 1\}$: LP-open facilities,
- $J_{\mathrm{frac}} = \{j \in \mathcal{J} : 0 < y_j^{\mathrm{LP}} < 1\}$: LP-fractional facilities,
- $J_0 = \{j \in \mathcal{J} : y_j^{\mathrm{LP}} = 0\}$: LP-closed facilities.

The *kernel* is $\mathcal{K} = J_1 \cup J_{\mathrm{frac}}$, following Guastaroba and Speranza (2012).

Why the kernel predicts integer solutions.. The VUB constraints (4) force $x_{ji} \leq d_i y_j$: no demand can be routed to a facility without opening it, even fractionally, in the LP. A LP-closed facility ($y_j^{\text{LP}} = 0$) therefore carries no LP flow; the LP found it suboptimal to open facility j even fractionally. This is a useful signal: a facility that is LP-closed must overcome both fixed opening cost and capacity interactions before it becomes attractive in an integer solution. The computational evidence in Guastaroba and Speranza (2012) shows that the LP-defined kernel captures most facilities used in high-quality solutions on the large-scale benchmark families. This motivates the Kernel Search strategy: rather than solving the full m-facility MILP, start from $\mathcal{K}$ and test promising LP-closed facilities through restricted subproblems.

Reduced costs.. Let $\pi_i \geq 0$ be the dual variable for demand constraint (2) and $\alpha_j \leq 0$ the dual variable for capacity constraint (3). For the LP relaxation, the *reduced cost* of shipping variable x_{ji} is:

$$\bar{c}_{ji} = c_{ji} - \pi_i - \alpha_j. \tag{7}$$

At LP optimality, $\bar{c}_{ji} \geq 0$ for every variable at its lower bound ($x_{ji} = 0$) and $\bar{c}_{ji} = 0$ for every basic variable ($x_{ji} > 0$). A variable with large positive reduced cost is far from being optimal to include: the LP would need a much cheaper shipping cost, or a much better dual price, before it would use that arc. This signal guides both column generation (Section 3.3) and edge selection (Section 3.5).

Why column generation is natural.. The LP solution of the strong MSCFLP formulation is often sparse relative to the complete $|\mathcal{J}||\mathcal{I}|$ shipping-variable matrix. Many shipping variables are zero at LP optimality and have non-negative reduced cost. Column generation exploits this structure by leaving such variables outside the restricted master problem until their reduced cost indicates that they may improve the LP objective. Study 4.2 reports the observed active-column counts and checks LP agreement against the full formulation where the full LP can be solved.

3. Design Analysis and Algorithm

This section first recalls the G&S 2012 baseline (Section 3.1), then examines the three design aspects identified in the introduction (Section 3.2). The three modifications are presented in order of dependency: column-generation

LP (Section 3.3), feasibility-preserving stage transitions (Section 3.4), and flat-k arc selection (Section 3.5). Fix 3 is presented before Fix 2 because Proposition 1 is the structural reason the funnel's narrowing arc sets are safe to use: the fill size k can be reduced aggressively only because the incumbent is always feasible by construction regardless of how small k becomes. Section 3.6 presents the complete FKS-CG-WS algorithm.

3.1. Baseline Kernel Search (G&S 2012)

After solving the LP relaxation, facilities with positive LP dual ($\alpha_j^* > 0$) form the *kernel* $\mathcal{K} = J_1 \cup J_{\text{frac}}$; the remaining LP-closed facilities ($\alpha_j^* = 0$) form J_0 and are organized into *buckets*. G&S 2012 then proceeds as follows:

1. **Kernel MILP.** Solve MILP restricted to facilities in kernel $\mathcal{K}$ and edges $\mathcal{E} = \{(j,i) : j \in \mathcal{K},\ \bar{c}_{ji} \leq \gamma\}$, where $\gamma = \text{mean}\{\bar{c}_{ji} : j \in \mathcal{K},\ \bar{c}_{ji} > 0\}$ is the single global mean of the positive reduced costs over all kernel arcs. An objective cutoff z^H is imposed.
2. **Bucket expansion.** Sort J_0 by $\bar{c}_{y_j}$ ascending. Add the next $l_{\text{buck}} = |\mathcal{K}|$ facilities as bucket B_h, connect to clients with $\bar{c}_{ji} \leq \gamma$, and solve MILP($\mathcal{K} \cup B_h$) with a forcing constraint $\sum_{j \in B_h} y_j \geq 1$. Repeat for up to $\bar{N}_B = 3$ buckets.
3. **Kernel update.** If an improved solution opens facilities from the bucket, add them to the kernel. Facilities already in the kernel are removed after repeated non-selection, controlled by parameter p. An iterative variant restarts the solution phase when new facilities are added.

This design gives several MILP-side controls over running time: the number of buckets, their size, the removal rule, and the time limit per restricted MILP.

3.2. Design Analysis: Three Refinement Opportunities

Before presenting FKS, we examine three aspects of the G&S 2012 design where targeted modifications improve performance. Each corresponds directly to one component of FKS.

LP construction cost.. The full LP relaxation contains $|\mathcal{J}||\mathcal{I}|$ shipping variables. At optimality, the large majority carry positive reduced cost and are set to zero; they play no role in the LP solution and therefore provide no useful signal to KS. Constructing the full LP matrix is therefore unnecessary: column

generation recovers the same LP bound, facility partition, and reduced-cost rankings without ever adding the irrelevant variables. On the largest instances (e.g., $|\mathcal{J}| = 2{,}000$, $|\mathcal{I}| = 2{,}000$), the full LP solve can dominate total running time, making KS slower than necessary before the first MILP is even solved.

γ-threshold arc selection: size and bias.. G&S 2012 includes arc (j, i) in a restricted MILP whenever its reduced cost satisfies $\bar{c}_{ji} \leq \gamma$, where γ is a single global scalar — the mean of the positive reduced costs over all kernel arcs. Two concerns follow. First, the number of arcs admitted by γ depends on LP concentration and can vary widely across instances and capacity regimes, giving unpredictable subproblem sizes. Second, consider a bucket facility $j \in J_0$ (LP dual $\alpha_j^* = 0$). By equation (8), its arc reduced cost is $\bar{c}_{ji} = c_{ji} - \pi_i^* - \alpha_j^* = c_{ji} - \pi_i^*$: the capacity dual contributes nothing. For a kernel facility $j' \in J_1$ with $\alpha_{j'}^* > 0$, the same formula gives $\bar{c}_{j'i} = c_{j'i} - \pi_i^* - \alpha_{j'}^* < c_{j'i} - \pi_i^*$. The capacity dual reduces kernel arc reduced costs, making them easier to pass the γ threshold. Bucket arcs must clear a structurally higher hurdle with no capacity signal subtracted, so fewer bucket arcs qualify for any given γ. This treats bucket facilities less favorably than kernel facilities, giving them a systematically smaller arc set in every restricted MILP.

Cold-start MILP transitions.. G&S 2012 passes no information between consecutive restricted MILPs. When the arc set changes (between stages or between kernel and bucket MILPs), the incumbent solution from the previous subproblem may not be feasible in the next one, so the solver must find feasibility from scratch. In the multi-stage setting this is wasteful: a good solution is available but cannot be used because there is no guarantee it remains feasible after the arc set is modified.

FKS addresses each aspect in turn through the components described in the following sections.

3.3. Column Generation for the LP

Motivation.. The full LP over all $|\mathcal{J}| \times |\mathcal{I}|$ shipping variables can dominate KS runtime on large instances. The MSCFLP LP solution is typically sparse: only a small fraction of shipping variables carry positive flow at optimality. Column generation exploits this structure by starting with a small column set and adding only inactive variables that can improve the LP objective.

Column generation setup.. We keep all y_j variables in the model from the start (there are only $|\mathcal{J}|$ of them). Only the x_{ji} shipping variables are generated on demand.

The *restricted master problem* (RMP) at iteration t contains:

- All y_j variables with their opening costs f_j.
- A subset $\mathcal{A}^t \subseteq \mathcal{J} \times \mathcal{I}$ of active shipping columns with costs c_{ji}.
- Demand constraints (2): $\sum_{j:(j,i)\in\mathcal{A}^t} x_{ji} = d_i$ for each i (initially enforced via a small artificial variable to guarantee feasibility).
- Capacity constraints (3): $\sum_{i:(j,i)\in\mathcal{A}^t} x_{ji} \leq q_j y_j$ for each j.
- VUB constraints (4): $x_{ji} \leq d_i y_j$ for each $(j,i) \in \mathcal{A}^t$.

When column (j,i) is added to $\mathcal{A}^t$, its variable x_{ji} enters the demand constraint for client i, the capacity constraint for facility j, and a new VUB constraint.

Initialization.. We initialize $\mathcal{A}^0$ by selecting the $k_{\text{init}} = 30$ cheapest facilities (by shipping cost c_{ji}) for each client i. This gives $30 \times |\mathcal{I}|$ initial columns (120,000 for a 1,000 × 4,000 instance), which is typically sufficient to produce a feasible RMP.

Pricing step.. After solving the RMP at iteration t, let π_i be the dual of demand constraint i and α_j the dual of capacity constraint j. The reduced cost of any inactive shipping variable x_{ji} ($(j,i) \notin \mathcal{A}^t$) is:

$$\bar{c}_{ji} = c_{ji} - \pi_i - \alpha_j. \tag{8}$$

If $\bar{c}_{ji} < 0$, then adding x_{ji} to the RMP can decrease the LP objective. We add *all* inactive columns with $\bar{c}_{ji} < 0$ simultaneously, then re-solve. This is the "full pricing" variant of column generation. It is more expensive per iteration than pricing a single column, but it avoids long sequences of small RMP resolves and is effective when many columns have negative reduced cost in early iterations. When a shipping column is added, its VUB constraint is introduced at the same time. After convergence, reduced costs used for edge selection are taken from the final LP solution: active generated columns use the solver's reduced costs, while inactive columns use the final demand and capacity dual prices. The computational study checks the CG LP against the full LP whenever the full LP can be solved within the experimental protocol.

Termination.. When no inactive column has $\bar{c}_{ji} < 0$ and all artificial variables are zero, the current RMP solution is accepted as the LP solution passed to KS. This follows from LP duality: at the optimal RMP dual (π, α), every column with $\bar{c}_{ji} \geq 0$ can only increase the objective if added, so no ungenerated shipping column is attractive under the pricing oracle.

Vectorized implementation.. The pricing step (8) is a matrix subtraction. Let $C \in \mathbb{R}^{|\mathcal{J}|\times|\mathcal{I}|}$ be the shipping cost matrix. After solving the RMP, the reduced costs of all $|\mathcal{J}| \times |\mathcal{I}|$ pairs are:

$$\mathrm{RC} = C - \boldsymbol{\pi}^{\top}[\text{broadcasted}] - \boldsymbol{\alpha}[\text{broadcasted}].$$

All inactive entries of RC with value < 0 are added in one operation. For $|\mathcal{J}| =$ 1,000 and $|\mathcal{I}| =$ 4,000, this is a single NumPy operation on a $1{,}000 \times 4{,}000$ matrix and is small relative to the RMP solves in our target setting.

Convergence in practice.. For each reported instance where the full LP is solved, we compare the CG objective value, facility partition, and reduced-cost rankings with those obtained from the full LP. This check is important because the purpose of CG in FKS is acceleration, not a change in the information used to guide Kernel Search. Section 4.2 reports the resulting CG iteration counts, active columns, LP times, and full-LP comparisons.

Remark on branch-and-price.. Column generation applied to the LP relaxation does *not* make FKS a branch-and-price algorithm. Branch-and-price is an exact method that solves an LP at every node of a branch-and-bound tree (Barnhart et al., 1998). FKS solves the LP exactly once (via CG), then runs heuristic MILPs with no LP re-solve. The heuristic character of FKS is unchanged; CG is purely a speed-up for the single LP solve that KS already requires.

3.4. Feasibility-Preserving Stage Transitions

When an arc set changes between stages, arcs used by the incumbent may be absent from the new set, making the incumbent infeasible. The solver must then rediscover feasibility from scratch. We eliminate this by including all incumbent arcs in the next stage's edge set by construction.

Let (y^s, x^s) be the best solution found after Stage s. For Stage $s{+}1$ with parameter k_{s+1}, we define the edge set as:

$$\mathcal{E}^{\mathrm{inc}}_{k_{s+1}}(x^s) = \bigcup_{i\in\mathcal{I}} \Big(\underbrace{\{(j,i) : x^s_{ji} > 0\}}_{\text{incumbent arcs}} \cup \underbrace{\text{top-}k_{s+1} \text{ by } \bar{c}_{ji} \text{ in } \mathcal{K}}_{\text{reduced-cost fill}} \Big). \tag{9}$$

Each later stage therefore contains the incumbent arcs plus a reduced-cost fill set. The edge count is at most $k_{s+1}|\mathcal{I}|$ plus the number of incumbent arcs not already in the fill set.

Proposition 1 (Feasibility Preservation)**.** *Let (y^s, x^s) be a feasible solution to the Stage s restricted MILP with facility set $\mathcal{F}^s$ and edge set $\mathcal{E}^s$. Let the Stage $(s{+}1)$ facility set $\mathcal{F}^{s+1}$ contain every facility with $y^s_j = 1$. Let $\mathcal{E}^{s+1} = \mathcal{E}^{\text{inc}}_{k_{s+1}}(x^s)$ as in equation* (9)*. Then (y^s, x^s) is feasible in the Stage $(s{+}1)$ restricted MILP.*

Proof. By construction (9), for every client i and every facility j with $x^s_{ji} > 0$, the arc (j, i) belongs to $\mathcal{E}^{s+1}$. The assumption on $\mathcal{F}^{s+1}$ ensures every facility opened by y^s is available in the next restricted MILP. We verify the three constraint types:

1. **Demand.** (y^s, x^s) satisfies $\sum_j x^s_{ji} = d_i$ for every client i. The same equality holds in Stage $(s{+}1)$ because every arc used by x^s is present.
2. **Capacity.** (y^s, x^s) satisfies $\sum_i x^s_{ji} \le q_j y^s_j$. Since y^s and x^s are unchanged, this constraint is satisfied.
3. **Variable upper bound.** (y^s, x^s) satisfies $x^s_{ji} \le d_i y^s_j$ for every arc with $x^s_{ji} > 0$. These arcs are present in $\mathcal{E}^{s+1}$, so the VUB constraints are present and satisfied.

Therefore (y^s, x^s) is feasible in Stage $(s{+}1)$. □ □

Corollary 1. *When (y^s, x^s) is injected as a MIP start for Stage $(s{+}1)$, the solver receives a feasible incumbent by construction and can begin from a known primal bound.*

Remark 1. *Proposition 1 does not depend on the LP solution structure, the instance size, or the capacity ratio. It holds for any multi-stage matheuristic that retains incumbent facilities and builds edge sets as in equation* (9)*, making the feasibility-preserving construction a general design principle for Kernel Search algorithms.*

Practical effect.. Stage 1 uses `MIPFocus=1` (Gurobi's feasibility-first mode) because no warm start is yet available. Stages 2 and 3 inject the previous incumbent and use `MIPFocus=0` (balanced), because a primal bound is already known. This removes the need for later stages to rediscover feasibility after the arc set has been narrowed.

The feasibility guarantee is also the structural reason the flat-k funnel in the next section is safe to use: because Proposition 1 holds regardless of how narrow the fill set is, the fill size k can be reduced aggressively across stages without risking cold restarts.

3.5. Edge Selection: The Flat-k Criterion

Section 3.4 establishes that all incumbent arcs must be retained in the next stage's edge set. This section specifies the remaining arcs: the reduced-cost fill in equation (9). We use a *client-centric, flat-k* criterion: for each client i, include the k facilities with the lowest reduced cost $\bar{c}_{ji}$.

Stage 1 edge set.. Stage 1 has no incumbent, so the edge set is a pure reduced-cost selection:

$$\mathcal{E}_{k_1}^{\mathrm{RC}} = \bigcup_{i \in \mathcal{I}} \big\{ (j,i) : j \text{ is among the } k_1 \text{ lowest-}\bar{c}_{ji} \text{ facilities in } \mathcal{K} \big\}. \tag{10}$$

We use $k_1 = 50$, giving 50,000 edges on a 1,000 × 1,000 instance and scaling linearly with $|\mathcal{I}|$ regardless of LP behavior. Stages 2 and 3 use equation (9) with $k_2 = 20$ and $k_3 = 10$: the fill set shrinks while the incumbent arcs are always retained by Proposition 1.

Proposition 2 (Guaranteed subproblem size)**.** *For any MSCFLP instance, any LP solution, and any $k \geq 1$, the Stage 1 edge set satisfies $|\mathcal{E}_{k_1}^{\mathrm{RC}}| = k_1 \cdot |\mathcal{I}|$. This bound is exact and holds regardless of LP concentration, capacity ratio, or instance scale.*

The proof is immediate: exactly k_1 arcs are selected per client and clients are disjoint. The Stage 1 subproblem size is fully controlled through k_1 alone.

Why flat-k rather than a threshold γ?. Section 3.2 gives two reasons. First, the γ-threshold provides no guarantee on subproblem size. Second, it is structurally biased: bucket facilities have $\alpha_j^* = 0$, so their arc reduced costs receive no capacity-dual discount and face a higher effective hurdle than kernel-facility arcs under the same γ. The flat-k criterion eliminates both problems: it selects exactly k arcs per client and ranks all facilities by the same reduced-cost ordering, treating kernel and bucket facilities identically.

Why three stages?. The specific values $k_1 = 50$, $k_2 = 20$, $k_3 = 10$ are a practical choice, not a theoretically derived optimum. What matters is that the sequence is decreasing (so the fill set narrows across stages) and that the feasibility guarantee of Proposition 1 holds at each transition regardless of how small k becomes. A practitioner with more time or memory can raise all three values.

3.6. Full Algorithm

Algorithm 1 states FKS precisely. All parameters not listed in the algorithm follow G&S 2012 defaults. We use $k_1 = 50$, $k_2 = 20$, $k_3 = 10$ and a 300-second time limit per stage throughout. When a bucket solution improves the incumbent, any opened bucket facility is promoted into the kernel before the next stage; this keeps the incumbent's facility set available for the feasibility-preserving transition. In the pseudocode, $\mathcal{E}_k^{\text{inc}}(x; \mathcal{F})$ denotes the same incumbent-preserving flat-k construction as equation (9), applied over facility set $\mathcal{F}$.

Why three stages and these k values?. The specific values $k_1 = 50$, $k_2 = 20$, $k_3 = 10$ are a practical choice, not a theoretically derived optimum. The flat-k design is intentionally parameterized: a practitioner with more time or memory can raise all three values; a practitioner with tighter constraints can lower them. What matters structurally is only that the sequence is decreasing (so the subproblem narrows progressively) and that incumbent arcs are preserved at each transition (so feasibility is not broken). The values used here were chosen to give a broad first-stage coverage on the largest benchmark families ($k = 50$ gives 100,000 arcs on a $1{,}000 \times 2{,}000$ instance) while keeping later stages computationally tractable. Because every stage after the first begins from a feasible warm start (Proposition 1), later stages spend their time improving the incumbent rather than rediscovering feasibility.

Relation to G&S 2012.. FKS retains all structural components of G&S 2012: the same kernel definition, the same bucket expansion structure, and the same MILP formulation. The two changes are: (1) the LP is solved via CG rather than by constructing the full LP upfront, and (2) the restricted-MILP phase is replaced by a three-stage funnel with feasibility-preserving transitions. The computational study evaluates these changes separately before reporting the combined end-to-end effect.

Algorithm 1 Funnel Kernel Search (FKS)

Require: Instance $(\mathcal{J}, \mathcal{I}, f, q, d, c)$; stage parameters $(k_s, T_s)_{s=1}^{S}$
Ensure: Feasible integer solution (y^H, x^H)

LP phase: Solve LP via column generation $\rightarrow (x^{\mathrm{LP}}, y^{\mathrm{LP}}, \bar{c})$; $z^{\mathrm{LP}} \leftarrow$ LP objective
$\mathcal{K} \leftarrow J_1 \cup J_{\mathrm{frac}}$
Sort J_0 by $\bar{c}_{y_j}$ ascending $\rightarrow$ bucket list L
$z^H \leftarrow +\infty$; $x^{\mathrm{prev}} \leftarrow \emptyset$; $y^H \leftarrow \emptyset$
for $s = 1, 2, \ldots, S$ **do**
  Build edge set:
  if $s = 1$ **then**
    $\mathcal{E} \leftarrow \mathcal{E}_{k_1}^{\mathrm{RC}}$ ▷ top-k_1 by reduced cost per client
    `MIPFocus` $\leftarrow 1$ ▷ find any feasible incumbent quickly
  else
    $\mathcal{E} \leftarrow \mathcal{E}_{k_s}^{\mathrm{inc}}(x^{\mathrm{prev}})$ ▷ preserve incumbent arcs, fill to k_s
    `MIPFocus` $\leftarrow 0$ ▷ balanced; warm start already feasible
  end if
  Kernel MILP: $(z, y, x) \leftarrow \mathrm{MILP}(\mathcal{K}, \mathcal{E}, z^H, \emptyset, T_s, \texttt{warm} = (y^H, x^H))$
  if $z < z^H$ **then** $z^H, y^H, x^H, x^{\mathrm{prev}} \leftarrow z, y, x, x$
  end if
  if $s > 1$ **then** ▷ bucket expansion (stages 2+ only)
    for $h = 1, \ldots, \bar{N}_B$ **do**
      $B_h \leftarrow$ next $l_{\mathrm{buck}} = |\mathcal{K}|$ facilities from L
      if $B_h = \emptyset$ **then break**
      end if
      $\mathcal{E}' \leftarrow \mathcal{E}_{k_1}^{\mathrm{inc}}(x^H; \mathcal{K} \cup B_h)$ ▷ flat-k bucket coverage plus incumbent arcs
      $(z, y, x) \leftarrow \mathrm{MILP}(\mathcal{K} \cup B_h, \mathcal{E}', z^H, B_h, T_s, \texttt{warm} = (y^H, x^H))$
      if $z < z^H$ **then** $z^H, y^H, x^H, x^{\mathrm{prev}} \leftarrow z, y, x, x$; $\mathcal{K} \leftarrow \mathcal{K} \cup \{j \in B_h : y_j = 1\}$
      end if
    end for
  end if
end for
return (y^H, x^H)

4. Computational Experiments

The computational study uses component-wise comparisons to isolate each of the three design refinements introduced in Section 3.2. In each comparison, exactly one component differs between the two variants, so the contribution of that specific modification is measured without confounding. This structure directly mirrors the analysis in Section 3.2: each comparison targets one of the three design aspects in isolation.

4.1. Experimental Setup

Instances.. We use the large-scale MSCFLP benchmark families of Avella et al. (2008), also used by Guastaroba and Speranza (2012). The main study covers the public Test Bed A and Test Bed C families listed in Table 1. Test Bed A and Test Bed C share the same five facility-client scales but use independently drawn instances, providing a robustness check across instance variation within each scale.

Table 1: Large-scale benchmark families used in the computational study.

Test Bed	Family	$\lvert\mathcal{J}\rvert$	$\lvert\mathcal{I}\rvert$	N	Instance indices
A	p800-4400	800	4,400	30	1–30
A	p1000-1000	1,000	1,000	30	1–30
A	p1000-4000	1,000	4,000	30	1–30
A	p1200-3000	1,200	3,000	30	1–30
A	p2000-2000	2,000	2,000	30	1–30
C	p1000-1000	1,000	1,000	30	61–90

Algorithms compared.. We report four variants, all implemented in Python 3.11 with Gurobi 13.0.2, run on an Apple M3 Pro (12-core CPU, 18 GB RAM). Each instance is solved in an independent process; only one instance runs at a time, so the full memory and CPU are available to each run.

- **KS-LP**: our re-implementation of G&S 2012 using the full LP relaxation (Gurobi simplex). All parameters follow G&S 2012 defaults. KS-LP is the within-study baseline; comparing against it measures the combined effect of all three fixes.

- **KS-CG**: the same KS-LP restricted-MILP phase with the full LP replaced by column generation. Comparing KS-CG against KS-LP isolates Fix 1 (LP cost) alone.
- **KS-CG-WS**: KS-CG with feasibility-preserving stage transitions and warm starting added, but retaining the original γ-threshold arc selection. Comparing KS-CG-WS against KS-CG isolates Fix 3 alone.
- **FKS-CG-WS**: Funnel Kernel Search — KS-CG-WS with the γ-threshold replaced by the flat-k criterion, $(k_1, k_2, k_3) = (50, 20, 10)$. Comparing FKS-CG-WS against KS-CG-WS isolates Fix 2 alone. Comparing FKS-CG-WS against KS-LP gives the combined end-to-end picture.

Implementation details.. KS-LP uses the parameter choices of G&S 2012: the LP-defined initial kernel, bucket size $l_{\text{buck}} = |\mathcal{K}|$, at most three analyzed buckets ($\bar{N}_B = 3$), objective cutoffs, and the bucket-forcing constraint. FKS uses the same kernel and bucket logic but replaces the single restricted-MILP phase with the flat-k funnel described in Section 3. The restricted-MILP time limit is 300 seconds per subproblem for all variants. G&S 2012 allocated 900 seconds per subproblem (Guastaroba and Speranza, 2012) (Section 3.2), noting that "the software often finds quickly an optimal or near-optimal solution" and that the limit exists mainly as a safeguard. Modern hardware and Gurobi reduce per-subproblem solve times considerably relative to the CPLEX 12.2 environment of G&S 2012, so 300 seconds proves sufficient: our KS-LP recreation matches or slightly exceeds their published aggregate results on p1000-1000 (Section 4.7). Because all four variants share this 300-second limit, all within-study comparisons are unaffected by the difference from the original paper. Runs are recorded with instance name, test bed, LP solver, stage widths, LP time, restricted-MILP time, objective value, and gap.

Metrics.. The LP gap is

$$\text{gap}(\%) = \frac{z^H - z^{\text{LP}}}{z^{\text{LP}}} \times 100,$$

where z^H is the heuristic objective and z^{LP} is the LP lower bound from the same run. Because our LP lower bounds can differ slightly from those reported by G&S 2012 due to solver and hardware differences (see Section 4.2.1), we also report integer objectives for direct literature comparisons. Win/tie/loss counts use a tolerance of 0.01 cost units. Runtime is decomposed into LP time, restricted-MILP time, and total time whenever the breakdown is relevant.

4.2. Study 1: LP Bottleneck Removal

The first study measures how much of the KS setup time is consumed by the LP phase and how accurately column generation replicates the full LP solution. This is primarily a timing and fidelity check, not a heuristic-quality comparison.

Table 2 summarizes LP timing across all benchmark families. Two patterns stand out. First, CG LP always matches the full LP objective to machine precision: the two lower bounds agree in all reported significant figures on every completed instance, confirming that column generation supplies the same information to KS without altering its guidance. Second, the CG speedup grows sharply with instance size. For p1000-1000 (Test Bed A), the full LP averages 58.0 seconds and CG reduces this to 1.3 seconds (45.6×) using 3.4 iterations and activating only 3.3% of the 10^6 possible shipping arcs. For p2000-2000, the full LP averages 710.7 seconds and CG reduces this to 4.2 seconds, a 170.2× speedup. The active-arc percentage falls as the instance grows (3.8% for p800-4400 to 2.4% for p2000-2000), consistent with larger instances having sparser LP solutions and therefore more redundant columns to avoid.

Table 2: LP relaxation: full LP (Gurobi simplex) versus column-generation LP. Speedup is the ratio of average full-LP time to average CG-LP time. Active arcs reports the percentage of all $|\mathcal{J}| \times |\mathcal{I}|$ arcs variables present in the terminal CG LP. CG LP objectives match full LP objectives to machine precision on all completed instances.

Test Bed	Family	Full LP (s)	CG LP (s)	Speedup	CG iters	Active arcs (%)
A	p800-4400	276.5	5.9	46.7×	2.6	3.8
A	p1000-1000	58.0	1.3	45.6×	3.4	3.3
A	p1000-4000	409.1	6.9	59.3×	2.8	3.0
A	p1200-3000	389.3	4.5	85.8×	3.3	2.5
A	p2000-2000	710.7	4.2	170.2×	3.9	2.4
C	p1000-1000	8.8	0.7	13.5×	1.3	3.0

4.2.1. On LP Lower-Bound Differences Between Studies

Our full LP lower bounds for p1000-1000 match those of G&S 2012 exactly on tight-capacity instances (instances 1–5), but are slightly higher on looser-capacity instances (instances 16–30), by up to 25 cost units. We treat these differences as numerical and reporting differences across software generations rather than as an algorithmic improvement in the LP bound. The original

study used a different solver environment and reports rounded values, while our within-study comparisons use LP bounds computed in the same Gurobi environment for all three variants. The important validation for this paper is therefore internal: the CG LP reproduces our full LP lower bound to within 1.3×10^{-5} on all instances, confirming that column generation supplies the same LP information used by the full-LP KS baseline.

4.3. Study 2: Restricted-MILP Acceleration

The second study compares KS-CG and FKS-CG-WS on the restricted-MILP phase. Both variants receive identical CG LP output, so any difference in MILP time or solution quality is attributable to the flat-k arc selection and feasibility-preserving stage transitions combined. Table 3 reports results across all families; a separate component-wise breakdown isolating each fix individually is given in Section 4.5.

FKS-CG-WS never loses on solution quality: across all five Test Bed A families (150 instances total), FKS-CG-WS matches or improves KS-CG on every instance. The quality advantage is most pronounced on p1000-4000 (23W/7T/0L) and p2000-2000 (22W/8T/0L), where the flat-k arc selection appears to consistently deliver better facility combinations than the γ-threshold.

MILP-phase speedup varies by family and depends on how incumbents interact with stage transitions. On families where LP solutions are relatively concentrated (p1000-1000: 1.86×; p2000-2000: 2.18×), the funnel's narrowing edge sets reduce subproblem size at each stage and FKS-CG-WS is substantially faster. On families with more diffuse LP support (p800-4400: 0.94×; p1000-4000: 0.80×), incumbent-arc preservation fills later-stage edge sets with arcs from the Stage 1 solution, limiting the thinning effect of the funnel. In these cases FKS-CG-WS's MILP phase is slightly slower than KS-CG, but end-to-end FKS-CG-WS remains faster once the LP-phase savings are included (Table 4).

Test Bed C p1000-1000 (30 independent instances, indices 61–90) shows 1.76× MILP speedup with 7W/22T/1L. The single loss is instance 82, where KS-CG finds an objective 2.6 cost units lower (a 0.00007% relative difference), consistent with numerical noise rather than a systematic quality difference.

4.4. Study 3: End-to-End Performance

Table 4 reports end-to-end results across all families. FKS-CG-WS never loses on solution quality against KS-LP across all 150 Test Bed A instances:

Table 3: Restricted-MILP phase: KS-CG versus FKS-CG-WS on the same CG LP. MILP time excludes the shared LP phase. Speedup is KS-CG time divided by FKS-CG-WS time. W/T/L counts wins, ties, and losses for FKS-CG-WS relative to KS-CG by integer objective (±0.01 tolerance).

			Avg. MILP time (s)			
TB	Family	N	KS-CG	FKS-CG-WS	Speedup	W/T/L
A	p800-4400	30	513.7	546.3	0.94×	12/18/0
A	p1000-1000	30	80.6	43.4	1.86×	1/29/0
A	p1000-4000	30	444.3	555.0	0.80×	23/7/0
A	p1200-3000	30	508.8	481.3	1.06×	16/14/0
A	p2000-2000	30	479.2	219.5	2.18×	22/8/0
C	p1000-1000	30	125.1	71.2	1.76×	7/22/1

the W/T/L counts show wins or ties only. The same holds against KS-CG except for the single numerically negligible loss on Test Bed C instance 82 noted above.

The end-to-end speedup over KS-LP grows with instance size, driven primarily by the LP phase. For p1000-1000, where the full LP is already modest (58 s), the combined speedup is 3.08×. For p2000-2000, where the full LP averages 710.7 s and column generation reduces this to 4.2 s, the combined speedup reaches 5.32×.

The pattern across families also illustrates the interaction between the two phases. On p800-4400, FKS-CG-WS's MILP phase is slightly slower than KS-CG (0.94×), but the LP-phase saving of 270 s per instance still produces a 1.52× end-to-end gain over KS-LP. On p1000-4000, FKS-CG-WS's MILP phase is again slower than KS-CG (0.80×), yet end-to-end FKS-CG-WS is 2.01× faster than KS-LP because the LP saving (409 s full LP versus 6.9 s CG) dominates. On p2000-2000, both phases favor FKS-CG-WS simultaneously: the LP phase contributes a 170× speedup and the MILP phase contributes a further 2.18×, yielding the largest combined gain in the study.

4.5. Study 4: Component-wise Isolation of Fix 2 and Fix 3

Studies 1–3 compare FKS-CG-WS against KS-CG, which differs in two ways: it uses flat-k arc selection (Fix 2) and feasibility-preserving stage transitions (Fix 3). To isolate each fix, we compare against KS-CG-WS, which adds only the feasibility-preserving transitions to KS-CG while retaining the γ-threshold.

Table 4: End-to-end total time (LP + restricted-MILP) for all three variants. Speedup is KS-LP time divided by FKS-CG-WS time. W/T/L counts wins, ties, and losses for FKS-CG-WS relative to each baseline by integer objective (±0.01 tolerance). Solution quality (gap) is reported separately in Table 3.

			Avg. total time (s)				W/T/L (FKS-CG-WS vs)	
TB	Family	N	KS-LP	KS-CG	FKS-CG-WS	Speedup	KS-LP	KS-CG
A	p800-4400	30	838.3	519.6	552.3	1.52×	15/15/0	12/18/0
A	p1000-1000	30	137.9	81.8	44.7	3.08×	4/26/0	1/29/0
A	p1000-4000	30	1,131.7	449.9	561.9	2.01×	17/13/0	23/7/0
A	p1200-3000	30	1,048.6	513.3	485.8	2.16×	16/14/0	16/14/0
A	p2000-2000	30	1,189.3	483.6	223.7	5.32×	22/8/0	22/8/0
C	p1000-1000	30	185.1	125.8	71.8	2.58×	4/25/1	7/22/1

- KS-CG versus KS-CG-WS: measures Fix 3 (feasibility-preserving transitions) alone, with arc selection held constant.
- KS-CG-WS versus FKS-CG-WS: measures Fix 2 (flat-k arc selection) alone, with feasibility-preserving transitions held constant.

Results for p1000-4000 Test Bed A (30 instances) will be reported for this ablation because its size and capacity regime produce clear quality differences in Study 2. The comparison uses the same instance set, CG LP output, and 300-second restricted-MILP limit as the main study. We keep this ablation separate from Tables 3 and 4 because it requires the additional KS-CG-WS control run; those runs are not mixed into the aggregate tables until the complete family is available.

4.6. Study 5: Comparison Against Proven Optima

The final study evaluates solution quality against the best available results from the literature for exact methods. For Test Bed A, we use the proven optimal values reported by Caserta and Voß (2020), who solved 110 of the 150 instances to optimality. For Test Bed C, we use the proven optimal values and best-known upper bounds from Avella et al. (2021), who provide the current state-of-the-art for these harder instances. The gap is computed as $\text{gap}(\%) = 100 \times (z^H - z^*)/z^*$, where z^* is the proven optimal or best-known objective.

Table 5 reports the average and maximum gaps for the 110 Test Bed A instances with known optima. The results show that FKS-CG-WS is exceptionally effective, finding the proven optimal solution on 109 of 110 instances. The single non-optimal solution (for instance p2000-2000-17) has a gap of just 0.0007%, corresponding to an objective value less than one cost unit from the optimum. This confirms that the FKS design not only accelerates the search but also consistently guides it to solutions of very high quality.

In contrast, the baseline KS-LP and KS-CG methods, while still strong, show small but consistent gaps across all families. Notably, KS-CG produces a large outlier gap of 0.27% on one instance (p1000-4000-17), where FKS-CG-WS finds the optimum. This suggests the flat-k arc selection criterion is more robust than the γ-threshold.

Table 5: Gap (%) versus proven optimal solutions on 110 Test Bed A instances. FKS-CG-WS finds the proven optimum on 109 of 110 instances.

	KS-LP		KS-CG		FKS-CG-WS	
Family (N)	Avg. gap	Max gap	Avg. gap	Max gap	Avg. gap	Max gap
p800-4400 (21)	0.0021%	0.0246%	0.0010%	0.0055%	0.0000%	0.0000%
p1000-1000 (30)	0.0018%	0.0373%	0.0001%	0.0025%	0.0000%	0.0003%
p1000-4000 (21)	0.0029%	0.0173%	0.0211%	0.2726%	0.0000%	0.0000%
p1200-3000 (19)	0.0027%	0.0268%	0.0014%	0.0096%	0.0000%	0.0000%
p2000-2000 (19)	0.0026%	0.0136%	0.0012%	0.0088%	0.0000%	0.0007%
Overall (110)	**0.0024%**	**0.0373%**	**0.0047%**	**0.2726%**	**0.0000%**	**0.0007%**

4.7. Validating the KS-LP Baseline Against Published Results

Why a re-implementation is necessary.. The computational environments in G&S 2012 and this study differ in solver (CPLEX 12.2 versus Gurobi), hardware (Intel Xeon 2.27 GHz versus modern hardware), and per-subproblem time limit (900 seconds versus 300 seconds). These differences make direct comparisons of running times meaningless, and they mean we cannot use the published G&S timing or solution results as the control in our ablation study. Instead, we implement a faithful G&S-style baseline — same formulation, same parameter choices (γ, l_{buck}, $\bar{N}_B$, p), same kernel construction and bucket logic — and run it as KS-LP in our own environment. All proposed variants (KS-CG, KS-CG-WS, FKS-CG-WS) are then compared against this same KS-LP baseline, ensuring that solver, hardware, and time limit are identical across every comparison in the study.

How we know the re-implementation is fair.. The validity of this design depends entirely on whether our KS-LP recreation is a faithful and adequate representation of the original algorithm. If our re-implementation were systematically weaker than G&S's, the study would be unfairly easy: our new methods would beat a handicapped baseline. We therefore validate KS-LP against G&S's published results on the p1000-1000 Test Bed A family, using the same benchmark quantity G&S report: improvement over the Avella et al. (Avella et al., 2008) published upper bounds.

G&S report an average improvement of -0.13% over those bounds, with 26 improvements out of 30 instances. Our KS-LP achieves the same average improvement figure, improves 28 upper bounds, and ties the remaining 2 — despite using only one-third of their per-subproblem time budget. Because G&S report rounded optimality gaps rather than raw integer objectives, we also perform a secondary reconstruction check using

$$\tilde{z}^{H}_{\mathrm{BKS}} = \frac{z^{LB}_{\mathrm{BKS}}}{1 - \mathrm{Gap}_{\mathrm{BKS}}/100},$$

which gives the same aggregate conclusion: our KS-LP objective is slightly lower than the reconstructed B-KS average (treated as an approximate consistency check only, given the rounded inputs).

Our KS-LP re-implementation therefore produces *at least as good* solution quality as the published G&S results on this family. This confirms that KS-LP is a fair, non-handicapped representation of the original algorithm, and that any gains attributed to KS-CG, KS-CG-WS, or FKS-CG-WS in the following studies represent genuine improvements over a credible baseline. Detailed per-instance results appear in Appendix Appendix A.

5. Conclusion

Kernel Search for the large-scale MSCFLP is effective because it uses the LP relaxation to identify a small set of promising facilities and arcs before solving restricted MILPs. The same dependence on the LP, however, exposes a bottleneck on the largest instances: obtaining the LP information can itself become expensive. This paper addresses that bottleneck directly by solving the KS LP relaxation through column generation. The intention is not to alter the heuristic guidance, but to obtain the LP-derived bound, facility partition, and reduced-cost signal at lower setup cost.

The second contribution concerns the restricted-MILP phase. The original KS framework already contains several speed controls, such as limiting bucket exploration and removing unpromising facilities from the kernel. Those controls reduce MILP effort but naturally introduce a speed-quality trade-off. FKS instead organizes the restricted search as a flat-k funnel. The flat-k rule gives predictable base subproblem sizes, while preserving all incumbent arcs and facilities between consecutive stages guarantees that the previous solution is feasible in the next restricted MILP. This gives a simple structural basis for warm-starting later stages after the edge set has been narrowed.

The broader message is diagnostic. Matheuristic improvements benefit from examining the existing method's design choices carefully before proposing new components. For G&S 2012 KS on the MSCFLP, a close reading surfaces three design aspects that can each be refined — avoidable LP cost, biased arc selection, and cold-start MILP transitions — each with a targeted, independently verifiable modification. The computational study is organized around this structure: each comparison changes exactly one component, making it possible to attribute improvements to a specific design decision rather than to the aggregate effect of the method.

Several extensions remain natural. First, completing the remaining Test Bed C families will provide a broader robustness check across facility-client scales and capacity regimes. Second, per-client adaptive budgets could allocate more arcs to clients with diffuse LP support and fewer arcs to clients with concentrated support while retaining the incumbent-preservation guarantee. Third, the feasibility-preservation principle may be useful in other Kernel Search settings whenever restricted subproblems are built by selecting subsets of variables used by an incumbent solution.

References

Avella, P., Boccia, M., Mattia, S., Rossi, F., 2021. Weak flow cover inequalities for the capacitated facility location problem. European Journal of Operational Research 289, 485–494.

Avella, P., Boccia, M., Sforza, A., Vasil'ev, I., 2008. An effective heuristic for large-scale capacitated facility location problems. Journal of Heuristics 15, 597–615.

Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance,

P.H., 1998. Branch-and-price: Column generation for solving huge integer programs. Operations Research 46, 316–329.

Caserta, M., Voß, S., 2020. A general corridor method-based approach for capacitated facility location. International Journal of Production Research 58, 3855–3880.

Daskin, M.S., 2011. Network and Discrete Location: Models, Algorithms, and Applications. 2nd ed., Wiley, Hoboken, NJ.

Fischetti, M., Ljubić, I., Sinnl, M., 2016. Benders decomposition without separability: A computational study for capacitated facility location problems. European Journal of Operational Research 253, 557–569.

Görtz, S., Klose, A., 2012. A simple but usually fast branch-and-bound algorithm for the capacitated facility location problem. INFORMS Journal on Computing 24, 597–610.

Guastaroba, G., Speranza, M.G., 2012. Kernel search for the capacitated facility location problem. Journal of Heuristics 18, 877–917.

Klose, A., Drexl, A., 2005. Facility location models for distribution system design. European Journal of Operational Research 162, 4–29.

Laporte, G., Nickel, S., Saldanha da Gama, F., 2019. Location science .

Sridharan, R., 1995. The capacitated plant location problem. European Journal of Operational Research 87, 203–213.

Appendix A. Detailed Per-Instance Results

Tables A.6 and A.7 give per-instance results for the p1000-1000 and p800-4400 Test Bed A families respectively. Table A.6 gives per-instance results for the p1000-1000 Test Bed A family. The LP lower bound z^{LP} is shared by all three variants (the CG LP reproduces the full LP objective to machine precision on every instance). Objective values and z^{LP} allow the reader to verify the reported gap directly as $(z^{H} - z^{\mathrm{LP}})/z^{\mathrm{LP}} \times 100$. Time is total wall-clock seconds (LP + restricted-MILP). A bold FKS-CG-WS objective indicates a strictly better solution than KS-LP.

Table A.6: Per-instance results: p1000-1000, Test Bed A. z^{LP}: LP lower bound (identical for all three variants). Objective columns report integer objective values; time columns report total wall-clock seconds (LP + MILP). Bold FKS-CG-WS objective indicates a strictly better solution than KS-LP.

		KS-LP		KS-CG		FKS-CG-WS	
Instance	z^{LP}	Obj.	Time	Obj.	Time	Obj.	Time
p1000-1000-1	920,490.9	920,505.2	117.0	920,505.2	64.6	920,505.2	42.4
p1000-1000-2	920,809.1	920,834.2	88.4	920,834.2	37.2	920,834.2	10.5
p1000-1000-3	931,266.1	931,305.4	109.5	931,305.4	69.1	931,305.4	17.3
p1000-1000-4	917,924.5	917,951.7	114.7	917,951.7	65.9	917,951.7	10.2
p1000-1000-5	915,609.1	915,635.4	118.1	915,635.4	64.7	915,635.4	12.2
p1000-1000-6	530,717.7	530,737.5	245.9	530,737.5	138.6	530,737.5	30.9
p1000-1000-7	570,761.3	570,792.9	340.0	570,792.9	292.2	570,792.9	40.7
p1000-1000-8	573,812.7	573,827.8	131.3	573,827.8	61.3	573,827.8	10.7
p1000-1000-9	556,659.9	556,686.4	167.6	556,686.4	91.3	556,686.4	14.0
p1000-1000-10	543,588.6	543,599.8	149.4	543,599.8	57.5	543,599.8	7.2
p1000-1000-11	350,058.3	350,069.3	115.6	350,069.3	76.5	350,069.3	13.2
p1000-1000-12	311,493.4	311,516.2	131.3	311,516.2	52.7	311,516.2	12.5
p1000-1000-13	331,134.3	331,149.2	119.3	331,149.2	48.2	331,149.2	11.1
p1000-1000-14	334,602.1	334,629.8	164.3	334,630.9	74.6	**334,623.6**	14.9
p1000-1000-15	347,379.5	347,407.3	144.6	347,407.3	72.8	347,407.3	15.1
p1000-1000-16	95,940.6	95,959.2	113.2	95,959.2	37.8	95,959.2	15.4
p1000-1000-17	93,969.0	93,998.8	92.6	93,998.8	38.1	93,998.8	13.6
p1000-1000-18	95,711.8	95,741.1	115.9	95,737.2	77.6	**95,737.2**	30.4
p1000-1000-19	91,028.2	91,057.3	128.9	91,057.3	58.2	91,057.3	22.2
p1000-1000-20	97,265.2	97,280.1	79.7	97,280.1	34.7	97,280.1	15.5
p1000-1000-21	49,845.2	49,884.8	89.7	49,884.8	23.6	49,884.8	16.6
p1000-1000-22	51,086.2	51,137.5	147.4	51,137.5	95.1	51,137.5	46.3
p1000-1000-23	50,015.9	50,066.5	118.7	50,060.8	43.9	**50,060.8**	30.3
p1000-1000-24	50,437.8	50,504.3	178.8	50,504.3	72.8	50,504.3	55.8
p1000-1000-25	48,247.2	48,338.1	282.4	48,338.1	343.2	48,338.1	283.1
p1000-1000-26	27,364.8	27,491.0	117.3	27,491.0	78.0	27,491.0	136.7
p1000-1000-27	26,518.5	26,586.9	118.4	26,586.9	79.9	26,586.9	94.3
p1000-1000-28	26,471.6	26,563.9	98.2	26,563.9	72.7	26,563.9	61.6
p1000-1000-29	27,758.3	27,869.8	112.0	27,859.4	80.8	**27,859.4**	182.7
p1000-1000-30	26,740.6	26,825.2	85.5	26,825.2	50.0	26,825.2	73.8
Average			137.9		81.8		44.7

Table A.7: Per-instance results: p800-4400, Test Bed A. z^{LP}: LP lower bound (identical for all three variants). Objective columns report integer objective values; time columns report total wall-clock seconds (LP + MILP). Bold FKS-CG-WS objective indicates a strictly better solution than KS-LP. Instances 1–15 are tight-capacity ($r \leq 2$); instances 16–30 are loose-capacity ($r \geq 3$).

		KS-LP		KS-CG		FKS-CG-WS	
Instance	z^{LP}	Obj.	Time	Obj.	Time	Obj.	Time
p800-4400-1	745,124.2	745,157.6	739.8	745,157.6	146.0	745,157.6	90.4
p800-4400-2	742,319.9	742,345.9	690.7	742,345.9	474.4	742,345.9	98.5
p800-4400-3	747,233.2	747,284.7	601.1	747,295.7	366.1	747,284.7	74.3
p800-4400-4	754,867.3	754,884.2	479.5	754,884.2	272.6	754,884.2	69.2
p800-4400-5	749,451.5	749,498.3	769.0	749,490.7	549.1	**749,490.7**	79.6
p800-4400-6	432,514.1	432,597.8	758.4	432,597.8	530.0	**432,588.2**	288.5
p800-4400-7	447,371.0	447,436.8	1149.8	447,436.8	555.3	447,436.8	254.2
p800-4400-8	468,666.4	468,724.7	837.7	468,724.7	554.1	468,724.7	130.7
p800-4400-9	424,817.2	424,888.3	803.8	424,888.3	514.9	**424,876.0**	116.7
p800-4400-10	441,145.9	441,234.7	783.3	441,234.7	472.2	**441,218.5**	135.3
p800-4400-11	290,454.5	290,562.1	832.0	290,562.1	461.0	**290,546.2**	139.8
p800-4400-12	292,151.3	292,220.8	733.5	292,220.8	421.9	**292,210.1**	194.1
p800-4400-13	304,467.3	304,561.7	828.3	304,561.7	498.4	304,561.7	275.4
p800-4400-14	304,332.3	304,417.4	827.6	304,417.4	458.6	304,417.4	203.4
p800-4400-15	282,818.8	282,870.4	820.4	282,870.4	341.3	282,870.4	78.9
p800-4400-16	100,510.0	100,662.7	663.8	100,662.7	419.9	100,662.7	704.9
p800-4400-17	99,790.5	99,985.7	1898.2	100,018.3	667.5	**99,966.9**	1161.5
p800-4400-18	103,716.6	103,945.4	1244.9	103,962.2	703.4	**103,921.1**	1016.1
p800-4400-19	104,544.4	104,689.0	783.8	104,689.0	468.5	104,689.0	743.7
p800-4400-20	102,307.9	102,456.4	576.2	102,453.9	485.0	**102,453.9**	374.0
p800-4400-21	72,364.3	72,593.0	842.5	72,558.3	841.3	**72,539.1**	1236.7
p800-4400-22	70,552.4	70,829.3	1413.2	70,823.9	910.0	**70,823.9**	1432.1
p800-4400-23	72,483.0	72,796.8	1540.4	72,773.3	578.9	**72,773.3**	1712.2
p800-4400-24	72,665.4	73,032.5	1423.5	73,018.0	999.5	**73,005.6**	1611.8
p800-4400-25	74,205.7	75,091.9	416.3	74,540.5	1138.9	**74,537.5**	1530.6
p800-4400-26	59,451.0	59,499.3	227.6	59,499.3	269.1	59,499.3	453.3
p800-4400-27	58,738.3	58,866.6	394.8	58,852.1	503.9	**58,852.1**	481.0
p800-4400-28	59,954.1	60,063.0	1327.7	60,063.0	559.7	60,063.0	962.4
p800-4400-29	57,817.3	57,936.4	491.0	57,938.5	276.9	57,936.4	692.3
p800-4400-30	57,973.0	58,041.4	251.0	58,041.4	150.0	58,041.4	226.0
Average			838.3		519.6		552.3